

ECORIDE

Dossier de gestion de projet et documentation technique

Projet de développement web — Application de covoiturage écologique

Ce document présente l'organisation du projet, les choix techniques, l'architecture, les bases de données, la sécurité, le déploiement et les principaux éléments nécessaires à la compréhension et à la maintenance de l'application.

PHP • MVC/POO • MySQL • MongoDB Atlas • Docker • GitHub • Render • Aiven

1. Gestion de projet

Le projet EcoRide est organisé selon une méthode agile simplifiée, avec un suivi des tâches sous forme de tableau Kanban. Cette organisation permet de planifier les développements, suivre les fonctionnalités, prioriser les corrections et préparer la remise.

1.1 Organisation du travail

- Identification des besoins fonctionnels et techniques.
- Découpage du projet en fonctionnalités et tâches.
- Suivi avec un tableau Kanban.
- Développement et tests.
- Corrections et vérifications finales.
- Versionnement Git et intégration GitHub.
- Préparation du déploiement et de la documentation.

1.2 Tableau Kanban

Le tableau de suivi distingue notamment les tâches terminées, les éléments en cours de finalisation et les vérifications à effectuer avant la remise. Le Kanban est fourni séparément dans le dossier **docs**.

2. Documentation technique

2.1 Réflexions technologiques

EcoRide est développé en PHP selon une architecture MVC et une programmation orientée objet. Le projet utilise MySQL pour les données métier, MongoDB Atlas pour la journalisation des activités, Composer pour les dépendances, JavaScript pour les interactions côté client et Docker pour la conteneurisation.

2.2 Configuration de l'environnement

- Visual Studio Code.
- XAMPP avec Apache et MySQL.
- MySQL local sur le port 3307.
- Composer pour les dépendances PHP.
- Extension PHP MongoDB et bibliothèque **mongodb/mongodb**.
- Docker pour la conteneurisation.
- Git et GitHub pour le versionnement.

3. Architecture MVC et programmation orientée objet

L'application est organisée selon une architecture MVC et une programmation orientée objet. Le code utilise l'espace de noms **Natom\Ecoride** et l'autoloading PSR-4 de Composer.

3.1 Principaux composants

- **app/Core/** : composants cœur tels que Database, Router et Controller.
- **app/Controllers/** : traitement des requêtes et logique applicative.
- **app/Models/** : accès et manipulation des données.
- **views/** : affichage des pages et gabarits.
- **public/** : point d'entrée et ressources publiques.

- **public/assets/js/** : scripts JavaScript.
- **docs/** : documentation et livrables.

3.2 Routage

Le point d'entrée public transmet les requêtes au routeur, qui les oriente vers les contrôleurs correspondants.

3.3 Connexion MySQL

La connexion MySQL utilise PDO et distingue l'environnement local de l'environnement de production. Les informations sensibles de production sont récupérées depuis les variables d'environnement.

4. Modèle de données et bases de données

4.1 Base relationnelle MySQL

MySQL constitue la base relationnelle principale d'EcoRide.

- **utilisateurs** : comptes, rôles et crédits.
- **vehicules** : véhicules déclarés.
- **preferences** : préférences des utilisateurs.
- **trajets** : trajets proposés.
- **participations** : participations aux trajets.
- **avis** : avis et état de validation.

4.2 MCD

Le MCD présente les principales entités de la base et leurs relations. Le diagramme MCD est fourni dans le dossier **docs**.

4.3 Base NoSQL — MongoDB Atlas

EcoRide utilise MongoDB Atlas comme base NoSQL complémentaire à MySQL, pour la journalisation des activités.

- MySQL conserve les données métier structurées.
- MongoDB conserve les données de journalisation.
- La connexion PHP utilise le client MongoDB et la bibliothèque **mongodb/mongodb**.
- La chaîne de connexion est conservée dans une variable d'environnement.

Répartition : données métier → MySQL ; journalisation → MongoDB.

5. Diagrammes et modélisation

5.1 Diagramme de cas d'utilisation

Le diagramme représente les interactions principales des rôles Utilisateur, Chauffeur, Employé et Administrateur.

- **Utilisateur** : inscription, connexion, recherche, participation, profil et avis.
- **Chauffeur** : proposition de trajet, participation, démarrage et clôture.
- **Employé** : validation/refus des avis et traitement des signalements.
- **Administrateur** : gestion des comptes et consultation des statistiques.

5.2 Diagramme de séquence

Le diagramme décrit notamment la recherche, la participation, la vérification des places et crédits, l'enregistrement de la participation, la mise à jour des places, la clôture du trajet et le processus d'avis.

6. JavaScript et traitement asynchrone

Le projet comporte du JavaScript côté client. Un appel asynchrone utilisant **fetch()** est présent dans **public/assets/js/admin.js**.

L'espace administrateur transmet une action au serveur en méthode POST avec FormData et utilise **await fetch()**, permettant de traiter l'action sans rechargement complet de la page.

7. Sécurité

- Hashage des mots de passe.
- Protection des routes.
- Vérification des rôles.
- Requêtes préparées avec PDO.
- Validation des données côté serveur.
- Variables sensibles de production conservées dans l'environnement.
- Gestion des dépendances PHP avec Composer.

8. Comptes de démonstration

Utilisateur : utilisateur@example.com / utilisateur123

Employé : employe@ecoride.fr / employe123

Administrateur : admin@ecoride.fr / admin123

9. Déploiement

9.1 Développement local

En local, l'application est exécutée avec XAMPP. Apache sert l'application PHP et MySQL fournit la base relationnelle locale sur le port 3307.

URL locale : <http://localhost:8080/ecoridestudi/ecoride/public/>

9.2 GitHub

Dépôt : <https://github.com/Nathan79300/ecoridestudi>

9.3 Render

L'application est déployée sur Render à partir du dépôt GitHub.

Application : <https://ecoridestudi.onrender.com>

9.4 Aiven

Aiven héberge la base MySQL de production. La connexion utilise les paramètres du service et SSL. Le mot de passe est conservé dans une variable d'environnement.

9.5 MongoDB Atlas

MongoDB Atlas fournit la base NoSQL. La connexion utilise une variable d'environnement et les informations d'authentification ne sont pas écrites en clair dans le code.

10. Conteneurisation avec Docker

Docker permet de disposer d'un environnement d'exécution reproductible.

- Extension MongoDB activée dans l'environnement Docker.
- Extension ZIP activée pour Composer.
- Dépendances PHP installées via `composer.json` et `composer.lock`.
- Le dossier `vendor/` reste exclu du versionnement Git.

11. Fonctionnement métier principal

11.1 Système de crédits

- Chaque utilisateur possède un solde de crédits.
- Les crédits initiaux sont attribués selon les règles du projet.
- La proposition d'un trajet applique le coût prévu.
- La participation entraîne le débit correspondant.
- Après validation du trajet, les crédits destinés au chauffeur sont traités.
- Une annulation peut entraîner le remboursement prévu.

11.2 Gestion des trajets

- Le chauffeur peut proposer un trajet avec un véhicule enregistré.
- Les places disponibles sont suivies lors des participations.
- Le chauffeur peut démarrer puis clôturer le trajet.
- La fin du trajet déclenche le processus de validation et d'avis.

11.3 Avis

- Le passager peut déposer un avis après le trajet.
- L'avis est soumis au contrôle.
- L'employé peut valider ou refuser l'avis.

12. Responsive Design

L'interface est conçue pour s'adapter aux différents formats d'écran grâce notamment aux Media Queries CSS.

- Ordinateur de bureau
- Tablette
- Mobile

13. Structure du projet

- **app/** : architecture MVC/POO, Core, contrôleurs et modèles.
- **public/** : point d'entrée et ressources publiques.
- **views/** : pages, vues et gabarits.
- **public/assets/js/** : scripts JavaScript.
- **docs/** : documents de l'ECF et documentation.
- **composer.json** : dépendances PHP.
- **composer.lock** : versions verrouillées.

- **Dockerfile** : conteneurisation.
- **.gitignore** : exclusions Git.

14. Organisation des livrables

Les documents nécessaires à la présentation du projet sont regroupés dans le dossier **docs**.

- Charte graphique.
- Diagramme de cas d'utilisation.
- Diagramme de séquence.
- MCD.
- Kanban.
- Manuel d'utilisation.
- Script SQL de la base.
- Dossier de gestion et documentation technique.

Le **README.md** reste à la racine du projet et présente les informations nécessaires à son installation et à sa compréhension.

15. Procédure de mise en place

15.1 En local

- Installer XAMPP et démarrer Apache et MySQL.
- Vérifier le port 3307.
- Placer le projet dans htdocs.
- Importer le script SQL.
- Installer les dépendances Composer.
- Vérifier les paramètres locaux.
- Accéder à l'URL locale.

15.2 En production

- Le code est versionné sur GitHub.
- Render construit et déploie l'application.
- Les variables d'environnement sont configurées dans Render.
- Aiven fournit MySQL.
- MongoDB Atlas fournit la base NoSQL.
- Docker fournit l'environnement conteneurisé.

16. Conclusion

EcoRide est une application web structurée en MVC/POO, versionnée avec Git, conteneurisée avec Docker et déployée avec GitHub, Render et Aiven. MySQL gère les données métier tandis que MongoDB Atlas fournit une solution NoSQL dédiée à la journalisation. Le projet intègre également JavaScript avec un traitement asynchrone via `fetch()`, une gestion multi-rôles, un système de crédits, des avis soumis à validation et une interface responsive.